

PARALLEL SUBDIVISION FOR SYNTHETIC TERRAIN MODELLING

Ian J. Curington
Floating Point Systems
Apex House, London Road
Bracknell, Berkshire RG12 2TE

ABSTRACT

A parallel program transformation is described for high performance synthetic terrain data generation. The 3-dimensional terrain grid undergoes binary subdivision along the two compass axes, while a random displacement control generates new altitude data. The terrain generation and hidden line plot is implemented on an FPS-5320 Array Processor.

INTRODUCTION

The generation of synthetic terrain data for visual scene simulation is of growing importance as special purpose real-time display processors are becoming prevalent. By exploiting parallel processing techniques the synthetic data may be generated at the same rate as it is plotted with both occurring within one frame interval. The implementation described in this paper uses an Array Processor with both parallel processors and vector architectures, where several subprograms may execute in parallel, with each gaining computational efficiency by the use of multiple pipelined arithmetic units.

Many papers have been published describing numerical methods and algorithms for the generation of random terrain, but most of these techniques have exploited recursive languages and used single instruction stream processors. Special hardware solutions to date have focussed on merging the data generation with the display algorithms in special purpose display processors [5]. This paper describes a general graphics device-independent method of achieving high performance through parallelism without recursion.

The mathematical foundations of random terrain are due to Mandelbrot [8], who has described a much larger class of functions called FRACTALS. The method employed to produce FRACTALS have very interesting mathematical properties, and are the focus of much current research [4,11,13]. FRACTAL functions while computationally expensive to evaluate, have properties absent from the technique presented here, although the results may appear similar [6]. The class of mathematical functions to which the technique presented here belongs has been termed GRAFTALS, [10] to distinguish it from pure FRACTALS.

The parallel subdivision method was first used to generate an animated film by Carpenter while with BOEING [14], and first described in 1982 [6]. This method of generating random terrain has proved to be a powerful tool, and many variations are currently under investigation both for use in animation and in the study of stochastic processes [2,7,10].

The terrain surface is represented by a two dimensional array

containing height values, or z-axis 'displacements, while the geometric transformation and data structures for the manipulation and display of the data has been covered in depth by several texts [1,9]. Every four adjacent grid points form a non-planar quadrilateral, which may be displayed directly by hidden-line methods, or divided into two or more triangles for shaded surface plotting [5].

SUBDIVISION METHOD

The subdivision method uses the symmetry of the regular grid to simplify the refinement. At each square defined by the four corner point values a new 3×3 array is generated by each subdivision step. The four corner points are used as seeds, or control values for the interpolated subdivision. Several methods for the subdivision are possible, [3,12] however the method chosen here benefits from computational simplicity, and the ease of creating the parallel method.

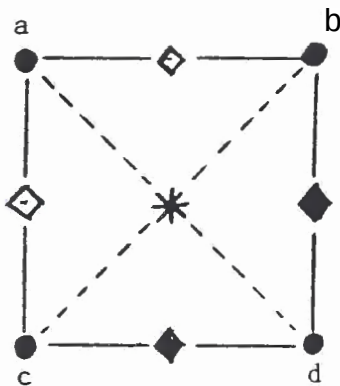


Figure 1a



Figure 1b



Figure 1c

2-point: $0.5 * (a + b) + \text{ran-disp} \Rightarrow \text{new side point.}$
 4-point: $0.25 * (a + b + c + d) + \text{ran-disp} \Rightarrow \text{new centre point.}$

FIGURE 1. Subdivision Linkage.

On the borders of the array only two seed points are available, so one new point is generated, as shown in figure 1b and 1c. In the interior of the array, all four points are used to generate 5 new points, as in figure 1a. During a sweep of the array, however, the kernel only generates three new points, shown as solid diamonds and the asterisk. The remaining two edge points are generated by previous steps which share control points. The array traversal moves in left to right, top to bottom order, sweeping through the array making successive refinements by powers of 2. The dimension of the result array is:
 $2 \times 2, 3 \times 3, 5 \times 5, 9 \times 9, 17 \times 17$, or 2^{n+1} for $n \ll 0$ through M ,
 where n is the iteration level, up to a maximum of M .

The vectorization of each refinement is across each row of the matrix, at first the vectors are quite short, but become longer as (2^n) as the refinement proceeds. The parallel processes are

assigned to alternating rows, up to the number of processors, then by a round-robin scheme. The control points for each iteration are global, so the shared edges are generated by both adjacent processors, but will arrive at the same answer due to the structure of the displacement algorithm. This is an explicit wave front technique, where all new values are computed once, with similar efficiency to previously described recursive algorithms with more complex data structures.

IMPLEMENTATION SOFTWARE

The terrain generation algorithm is coded in FORTRAN-77, with occasional Math Library routines, both running on the FPS-5320. The grid array is held locally in the System Common Memory, along with temporary arrays. The hidden line algorithm is integrated with the data generation, plotting the surface directly to an attached Bear Systems Frame store display.

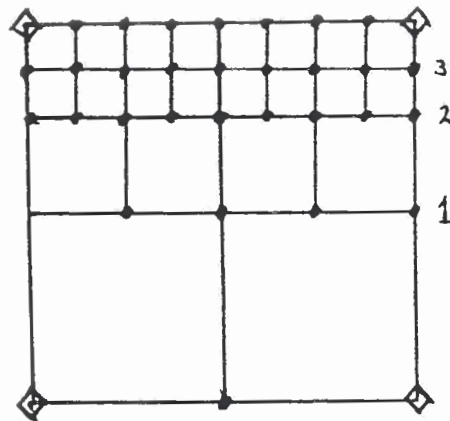


Figure 2. GRID Showing 3 levels of refinement from original corner points.

ROUGHNESS CONTROL

A global parameter between 0.0 and 1.0 is used to control the surface roughness, expressed as the local variance limit at each subdivision step. A value of 0 yields pure linear interpolation, a value of 0.1 yeilds a slight bumpy texture, while values approaching 1.0 yield very rough surface. The displacement value is scaled by both the global roughness factor and the distance between grid points. In this way the roughness factor scales uniformly at any resolution, and number of subdivision steps. The following illustration shows the effect of the roughness factor on the surface. The curves represent a vertical slice through the surface along a line of grid points.

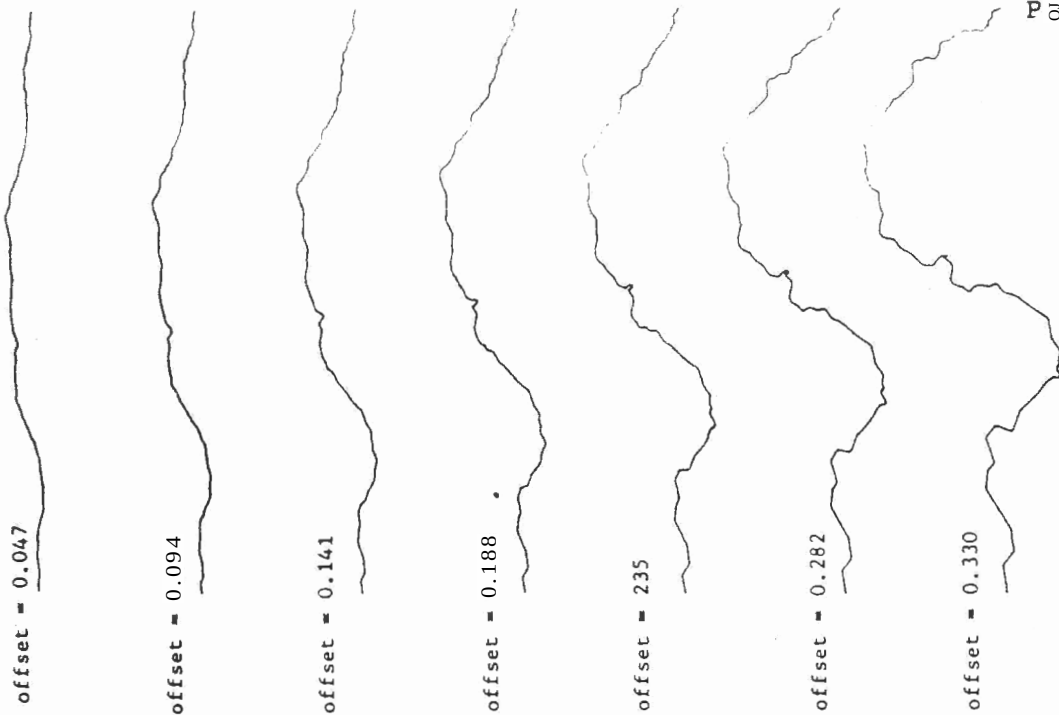


Figure 3. Roughness Control [3].

EDGE CONSISTANCY

The extension of this technique to arbitrary geometries such as polyhedra, triangular meshes, or solids, introduces a problem of edge consistency. The regular grid discussed above has the advantage that the same displacement will be calculated from either side of an edge, and the area of adjoining polygons are equal. This cannot be assured for arbitrary shapes and, in addition, recursive algorithms may not save displacement information across edges without the addition of special heuristics or data structures [2,6].

CONCLUSION

Terrain modelling is valuable in many areas, from flight simulation to algorithm testing. The ability to generate data at the same rate as it can be displayed is important to maintain flexibility in the real time process. The parallel architecture employed to generate the data is well matched to the parallel algorithm.

ACKNOWLEDGEMENTS

The author would like to thank Dr. Paul A. Daniels for the hidden-line plotting routine, and Dr. Ian King, Royal Naval Engineering College, Plymouth for inspiring discussions.

REFERENCES

- [1] Bowyer, A. and Woodwark J. "A Programmer's Geometry" Butterworths, publishers, London, 1983.

- [2] Coquillart, Sabine "Displaying Random Fields" Computer Graphics Forum, No. 4, North-Holland 1985.
- [3] Edwards, Gareth "Fractal Based Terrain Modelling" Proceedings of Computer FX 84, Wembley, Online Publications 1984.
- [4] Fellous, Granara, and Hourcade "Fractional Brownian Relief: An Exact Local Method" Proceedings of Eurographics 85, Nice, North-Holland Publishers, (C. Vandoni ed.) 1985.
- [5] Fornier, Milligan "Frame Buffer Algorithms for Stochastic Models" IEEE Computer Graphics and Applications, October 1985.
- [6] Fornier, Fussell, Carpenter "Computer Rendering of Stochastic Models" Communications of the ACM, Vol. 25, No. 6, June 1982.
- [7] Griffiths, J. G. "Table-Driven Algorithms for Generating Space-Filling Curves" Computer-Aided Design, Vol. 17, No. 1, Butterworth and Co. January 1985.
- [8] Mandelbrot, B. B. "The Fractal Geometry of Nature" Freeman 1983.
- [9] Rogers, D. F. "Procedural Elements for Computer Graphics" McGraw-Hill, 1985
- [10] Smith, Alvey Ray "Plants, Fractals, and Formal Languages" Computer Graphics (SIGGRAPH-84) Vol. 18, No. 3, July 1984.
- [11] Sorensen, Peter "Fractals" Byte Magazine, September 1984.
- [12] Wise, S. D. "The Use of Fractals in the Visualization of Random Terrain" Degree Thesis Report, R.N.E.C. Manadon, Plymouth U.K., March 1983.
- [13] Norton, Alan "Generation and Display of Geometric Fractals in 3-D" Computer Graphics (SIGGRAPH), Vol. 16, No. 3, July 1982.
- [14] Carpenter, L. "Vol Libra" 16mm Film, 1980.

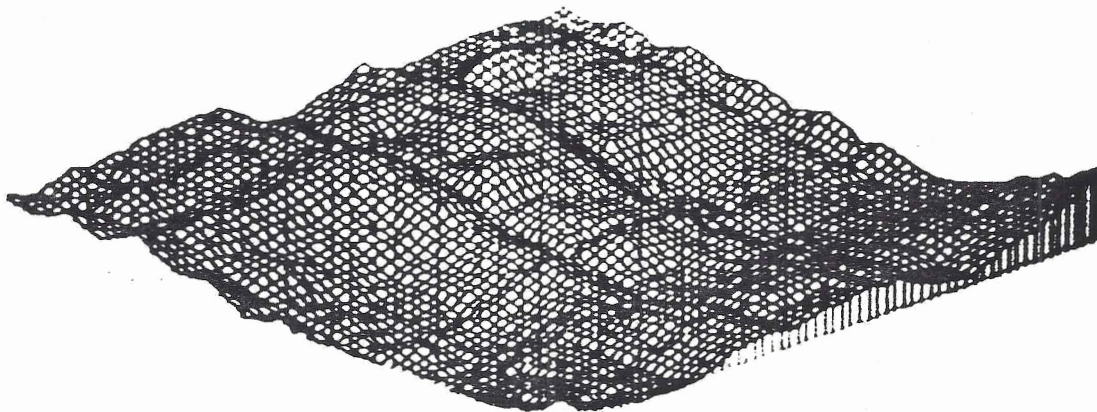


FIGURE 4. 3-Dimensional surface with low variance.

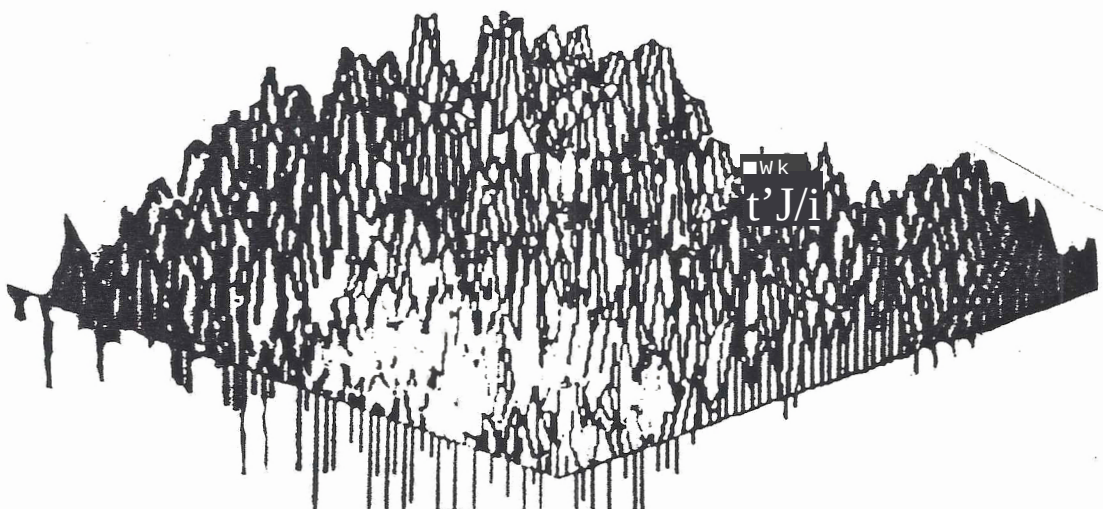


FIGURE 5. 3-Dimensional surface with high variance.